

Transfert Sécurisé de Fichiers

Table des matières

Transfert Sécurisé de Fichiers — Documentation Technique Complète

Table des matières

1. Vue d'ensemble du projet

Objectif

Structure du projet

2. Architecture générale

Séparation des responsabilités

3. Infrastructure PKI

Hiérarchie de certification

Processus de génération (`setup_pki.sh`)

Utilisation des fichiers PKI

4. Mécanismes cryptographiques

4.1 Chiffrement hybride

4.2 AES-256-CBC (`chiffreur_fichier_aes`)

4.3 RSA-OAEP (`chiffreur_cle_rsa`)

4.4 Hachage SHA-256 (`hash_sha256`)

4.5 Tableau récapitulatif des primitives cryptographiques

5. Flux de transfert complet

5.1 Vue macro du flux

5.2 Flux détaillé étape par étape

6. Protocole binaire

6.1 Structure du paquet

6.2 Assemblage côté client

6.3 Lecture sécurisée côté serveur

6.4 Protocole de réponse

7. Description des modules

7.1 `client.py`

7.2 `server.py`

7.3 `crypto_utils.py`

7.4 `demo.py`

8. Sécurité du transport (TLS)

8.1 Configuration TLS

8.2 Poignée de main TLS

8.3 TLS vs chiffrement applicatif

9. Fonctionnalités implémentées

Cryptographie

Infrastructure PKI

Transport sécurisé

Transfert de fichiers

Outils de développement

10. Fonctionnalités non implémentées

11. Analyse de sécurité

Points forts

Points à améliorer (pour un usage en production)

Modèle de menace couvert

12. Guide d'utilisation

Prérequis

Étape 1 — Génération de la PKI

Étape 2 — Démarrage du serveur

Étape 3 — Envoi d'un fichier (dans un autre terminal)

Étape 4 — Vérification du fichier reçu

Démonstration locale (sans réseau)

Résumé

Transfert Sécurisé de Fichiers — Documentation Technique Complète

Projet académique · USTHB Faculté Informatique **Auteurs** : Lisri Akram, Baatout Mohamed
Amine, Aeminee **Langage** : Python 3.10+ · **Outil cryptographique** : OpenSSL

Table des matières

1. [Vue d'ensemble du projet](#)
2. [Architecture générale](#)
3. [Infrastructure PKI](#)
4. [Mécanismes cryptographiques](#)
5. [Flux de transfert complet](#)
6. [Protocole binaire](#)
7. [Description des modules](#)
8. [Sécurité du transport \(TLS\)](#)
9. [Fonctionnalités implémentées](#)
10. [Fonctionnalités non implémentées](#)
11. [Analyse de sécurité](#)
12. [Guide d'utilisation](#)

1. Vue d'ensemble du projet

Secured Transfer est un système de transfert de fichiers sécurisé en réseau, développé en Python. Il combine plusieurs couches de sécurité afin de garantir la **confidentialité**, l'**intégrité** et l'**authenticité** des données transmises.

Objectif

Permettre l'envoi d'un fichier d'un client vers un serveur de manière totalement sécurisée, en utilisant :

- Le **chiffrement hybride** (AES + RSA) pour protéger le contenu
- Le **hachage SHA-256** pour vérifier l'intégrité
- **TLS 1.2+** pour sécuriser le canal réseau
- Une **PKI (Infrastructure à Clés Publiques)** pour l'authentification du serveur

Structure du projet

```
secured_trasfer/
├─ client.py          - Application cliente (chiffrement + envoi)
├─ server.py         - Application serveur (réception + déchiffrement)
├─ crypto_utils.py   - Fonctions cryptographiques (AES, RSA, SHA-256)
├─ demo.py           - Démonstration locale (sans réseau)
├─ setup_pki.sh      - Génération automatique de la PKI
├─ pki/
│  └─ ca/
│     ├── ca.key      - Clé privée CA (RSA 2048 bits)
│     └─ ca.crt       - Certificat CA (auto-signé, 10 ans)
│  └─ server/
│     ├── server.key  - Clé privée serveur (RSA 2048 bits)
│     ├── server.crt  - Certificat serveur (signé par CA, 365 jours)
│     └─ server_pub.pem - Clé publique serveur (utilisée par le client)
├─ received_files/   - Fichiers reçus et déchiffrés
└─ mon_fichier.txt   - Fichier de test
```

2. Architecture générale



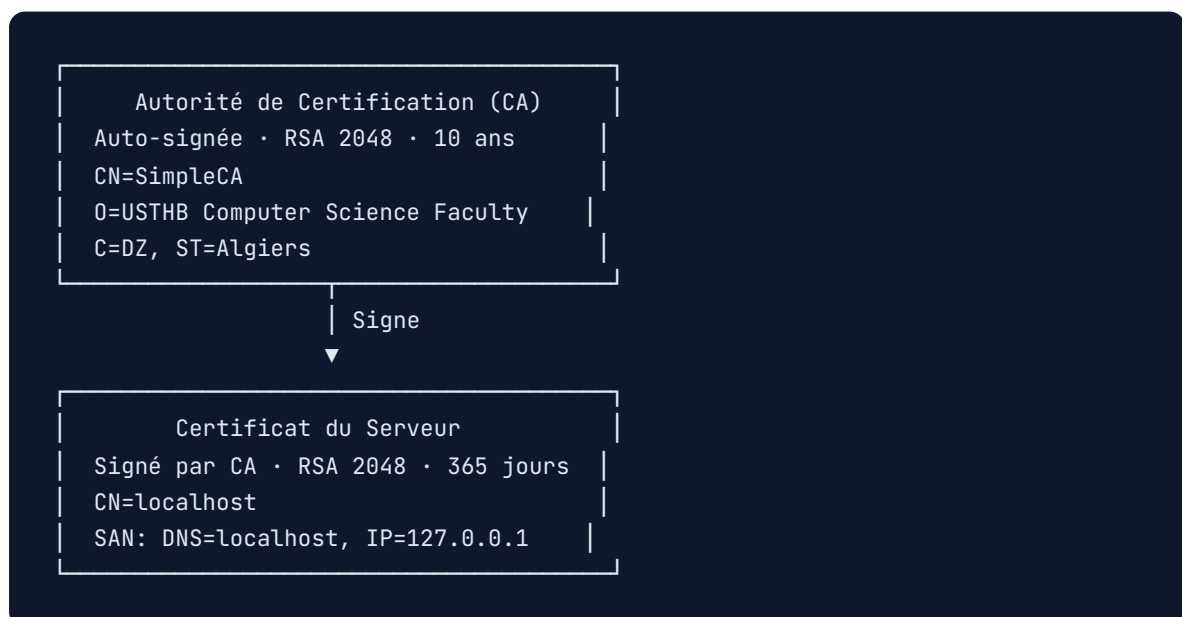
Séparation des responsabilités

Module	Rôle
<code>client.py</code>	Orchestration côté client : hachage, emballage, connexion TLS, envoi
<code>server.py</code>	Orchestration côté serveur : réception, déemballage, vérification, sauvegarde
<code>crypto_utils.py</code>	Couche cryptographique pure : AES, RSA, SHA-256 via OpenSSL
<code>demo.py</code>	Validation locale du cycle complet chiffrement/déchiffrement
<code>setup_pki.sh</code>	Génération automatisée de toute l'infrastructure PKI

3. Infrastructure PKI

La PKI (Public Key Infrastructure) est le fondement de toute la sécurité du projet. Elle est générée automatiquement par `setup_pki.sh`.

Hiérarchie de certification



Processus de génération (`setup_pki.sh`)

Étape 1 — Création de la CA

```
openssl genrsa -out pki/ca/ca.key 2048
openssl req -new -x509 -days 3650 -key pki/ca/ca.key -out pki/ca/ca.crt \
  -subj "/C=DZ/ST=Algiers/O=USTHB.../CN=SimpleCA"
```

Étape 2 — Génération du certificat serveur

```
# Clé privée du serveur
openssl genrsa -out pki/server/server.key 2048

# Demande de signature (CSR)
openssl req -new -key pki/server/server.key -out pki/server/server.csr \
  -subj "/C=DZ/ST=Algiers/O=SecureTransfer/CN=localhost"

# Extension SAN (Subject Alternative Name)
echo "[SAN]\nsubjectAltName=DNS:localhost,IP:127.0.0.1" > server_ext.cnf

# Signature par la CA
openssl x509 -req -days 365 \
  -in pki/server/server.csr \
  -CA pki/ca/ca.crt -CAkey pki/ca/ca.key \
  -extfile server_ext.cnf \
  -out pki/server/server.crt
```

Étape 3 — Export de la clé publique

```
openssl x509 -pubkey -noout \
  -in pki/server/server.crt \
  -out pki/server/server_pub.pem
```

Étape 4 — Vérification

```
openssl verify -CAfile pki/ca/ca.crt pki/server/server.crt
```

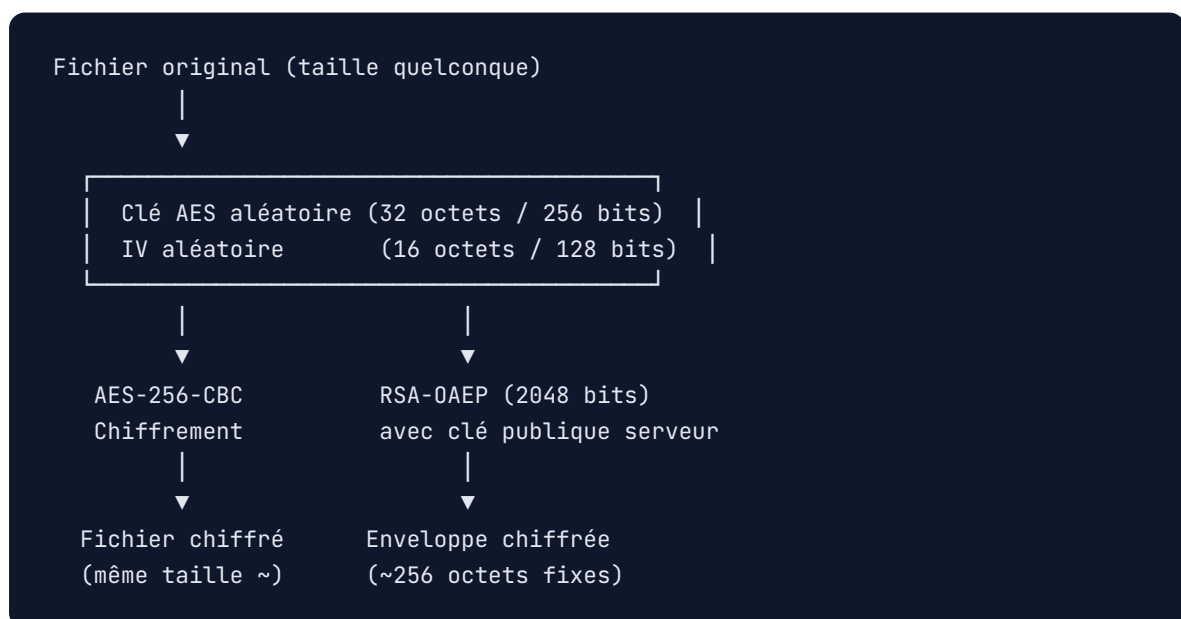
Utilisation des fichiers PKI

Fichier	Utilisé par	Usage
<code>pki/ca/ca.crt</code>	Client	Vérifier le certificat du serveur
<code>pki/server/server.crt</code>	Serveur	Présenter son identité TLS
<code>pki/server/server.key</code>	Serveur	Déchiffrer la clé AES (RSA privée)
<code>pki/server/server_pub.pem</code>	Client	Chiffrer la clé AES (RSA publique)

4. Mécanismes cryptographiques

4.1 Chiffrement hybride

Le projet utilise un **schéma de chiffrement hybride** qui combine : - La **rapidité** du chiffrement symétrique (AES) pour les données - La **sécurité** du chiffrement asymétrique (RSA) pour l'échange de clé



4.2 AES-256-CBC (`chiffrer_fichier_aes`)

```
# Génération de la clé et de l'IV
cle = os.urandom(32) # 256 bits aléatoires
iv  = os.urandom(16) # 128 bits aléatoires

# Appel OpenSSL
openssl enc -aes-256-cbc -nosalt
           -K {cle_hex}
           -iv {iv_hex}
           -in fichier_entree
           -out fichier_sortie
```

Paramètre	Valeur	Explication
Algorithme	AES-256-CBC	Norme NIST, mode CBC
Taille de clé	256 bits	Sécurité maximale de l'AES
IV	128 bits	Empêche les attaques par texte clair choisi
Padding	PKCS#7	Géré automatiquement par OpenSSL

4.3 RSA-OAEP (`chiffrer_cle_rsa`)

```
# Données à protéger : clé AES (32B) + IV (16B) = 48 octets
donnees = cle + iv # 48 octets

# Chiffrement avec la clé publique du serveur
openssl pkeyutl -encrypt
              -pubin
              -inkey pki/server/server_pub.pem
              -pkeyopt rsa_padding_mode:oaep
              -in donnees
              -out enveloppe_chiffree
```

Paramètre	Valeur	Explication
Algorithme	RSA-2048	Paire de clés asymétrique
Padding	OAEP	Plus sûr que PKCS#1 v1.5
Entrée	48 octets	clé AES + IV
Sortie	~256 octets	Enveloppe chiffrée fixe

4.4 Hachage SHA-256 (`hash_sha256`)

```
# Lecture par blocs de 8 Ko pour les grands fichiers
sha256 = hashlib.sha256()
with open(fichier, 'rb') as f:
    for bloc in iter(lambda: f.read(8192), b''):
        sha256.update(bloc)
return sha256.digest() # 32 octets binaires
```

Rôle : Vérifier que le fichier reçu est identique au fichier envoyé (intégrité).

4.5 Tableau récapitulatif des primitives cryptographiques

Composant	Algorithme	Taille	Norme	Rôle
Chiffrement données	AES-256-CBC	256 bits	NIST	Confidentialité du fichier
Vecteur d'initialisation	Aléatoire	128 bits	CBC	Unicité du chiffrement
Protection de la clé	RSA-OAEP	2048 bits	PKCS#1 v2.1	Chiffrement de la clé AES
Vérification intégrité	SHA-256	256 bits	FIPS 180-4	Détection d'altération
Sécurité du transport	TLS 1.2+	≥256 bits	RFC 5246	Chiffrement du canal
Certificat	X.509	RSA 2048	RFC 5280	Identité du serveur

5. Flux de transfert complet

5.1 Vue macro du flux



5.2 Flux détaillé étape par étape

Phase 1 — Configuration (une seule fois)

```
bash setup_pki.sh
└─ Génère CA (clé + certificat auto-signé)
└─ Génère serveur (clé + CSR + certificat signé par CA)
└─ Exporte la clé publique du serveur
└─ Vérifie la chaîne de certification
```

Phase 2 — Démarrage du serveur

```
python server.py
└─ Charge pki/server/server.crt et server.key
└─ Crée socket TCP sur 0.0.0.0:9443
└─ Enveloppe avec TLS 1.2 (SSLContext)
└─ Attend les connexions entrantes (boucle infinie)
```

Phase 3 — Envoi par le client

```
python client.py mon_fichier.txt
└─ Lit le fichier
└─ Calcule SHA-256(fichier) → hash 32 octets
└─ Génère clé AES aléatoire 256 bits
└─ Génère IV aléatoire 128 bits
└─ AES-256-CBC-Chiffre(fichier, clé, IV) → fichier_enc
└─ RSA-OAEP-Chiffre(clé||IV, serv_pub) → enveloppe
└─ Assemble paquet binaire
└─ Connecte en TLS à localhost:9443
└─ Vérifie certificat serveur via pki/ca/ca.crt
└─ Envoie taille_paquet (8 octets)
└─ Envoie paquet
└─ Reçoit confirmation
```

Phase 4 — Traitement par le serveur

```

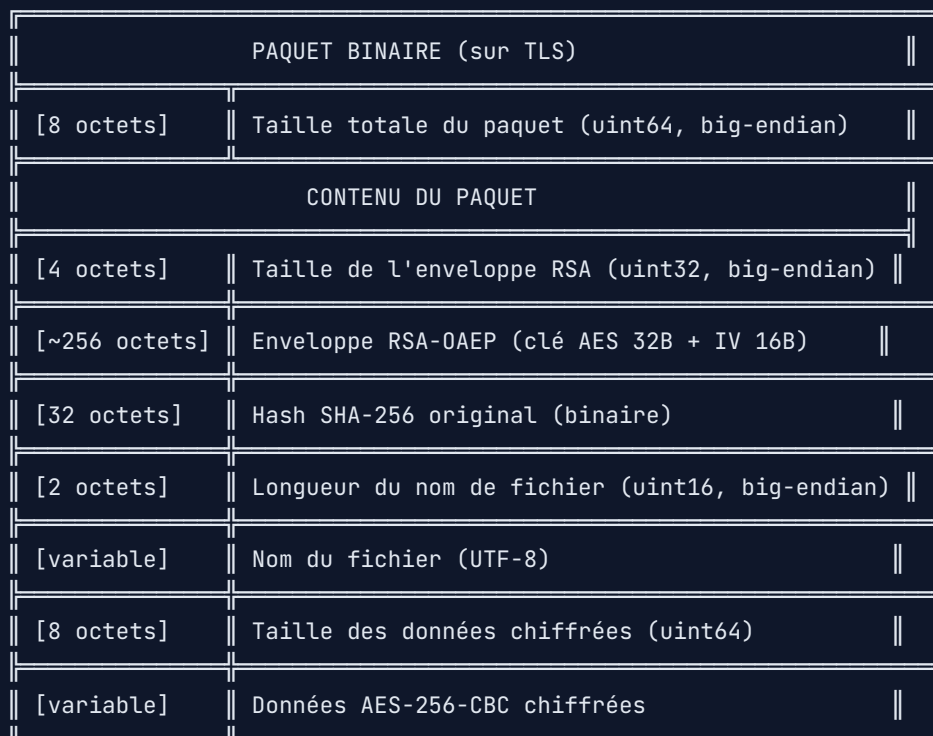
traiter_client(connexion, adresse)
└─ Reçoit taille_paquet (8 octets)
└─ Reçoit paquet complet (avec buffering 4Ko)
└─ Désassemble le paquet :
    └─ taille_env (4 octets) → extrait enveloppe
    └─ hash_original (32 octets)
    └─ taille_nom (2 octets) → extrait nom_fichier
    └─ taille_données (8 octets) → extrait données_chiffrées
└─ RSA-OAEP-Déchiffre(enveloppe, server.key) → (clé, IV)
└─ AES-256-CBC-Déchiffre(données_chiffrées, clé, IV) → fichier
└─ SHA-256(fichier) → hash_reçu
└─ Si hash_reçu = hash_original :
    └─ Sauvegarde dans received_files/YYYYMMDD_HHMMSS_nom.txt
    └─ Envoie "OK : fichier reçu et verifie"
└─ Sinon :
    └─ Envoie "ERREUR : hash incorrect"

```

6. Protocole binaire

6.1 Structure du paquet

Le client et le serveur communiquent via un **protocole binaire personnalisé** sur TLS :



6.2 Assemblage côté client

```
# Emballage binaire (struct Python)
import struct

taille_env = len(cle_chiffree)
nom_bytes = nom_fichier.encode('utf-8')

paquet = (
    struct.pack('>I', taille_env)          # 4B - taille enveloppe
    + cle_chiffree                        # ~256B - enveloppe RSA
    + hash_original                       # 32B - hash SHA-256
    + struct.pack('>H', len(nom_bytes))    # 2B - taille nom
    + nom_bytes                           # variable - nom fichier
    + struct.pack('>Q', len(donnees_enc))  # 8B - taille données
    + donnees_enc                         # variable - données AES
)

# Envoi avec préfixe de taille
sock.sendall(struct.pack('>Q', len(paquet))) # 8B taille totale
sock.sendall(paquet)                        # paquet complet
```

6.3 Lecture sécurisée côté serveur

```
def recevoir_donnees(sock, n):
    """Lit exactement n octets depuis le socket (avec buffering)"""
    donnees = b''
    while len(donnees) < n:
        morceau = sock.recv(min(4096, n - len(donnees)))
        if not morceau:
            raise ConnectionError("Connexion interrompue")
        donnees += morceau
    return donnees
```

Cette fonction garantit la réception complète même si les données arrivent en plusieurs fragments TCP.

6.4 Protocole de réponse

```
Serveur → Client :
"OK : fichier reçu et verifie"      (succès)
"ERREUR : hash incorrect"           (intégrité compromise)
"ERREUR"                            (autre erreur)
```

7. Description des modules

7.1 `client.py`

Rôle : Application cliente. Prend un fichier en argument, le chiffre et l'envoie au serveur.

Paramètre : `python client.py <chemin_fichier>`

Fonctions principales :

Fonction	Description
<code>envoyer_fichier(chemin)</code>	Orchestration complète : hash → AES → RSA → TLS → envoi

Configuration réseau :

```
HOTE = "localhost"
PORT = 9443
```

Flux interne :

```
envoyer_fichier(chemin_fichier)
1. hash_sha256(fichier)           → hash (32 octets)
2. chiffrer_fichier_aes(fichier)  → (données_enc, clé, iv)
3. chiffrer_cle_rsa(clé, iv, pub_key) → enveloppe
4. Assemblage paquet binaire
5. ssl.create_default_context()    → contexte TLS
6. ctx.load_verify_locations(ca.crt) → validation certificat
7. socket.connect(host, port)
8. sendall(taille) + sendall(paquet)
9. recv(256)                      → confirmation
```

7.2 `server.py`

Rôle : Application serveur. Écoute sur le port 9443, accepte les connexions, déchiffre et sauvegarde.

Paramètre : `python server.py` (aucun argument)

Fonctions principales :

Fonction	Description
<code>recevoir_donnees(sock, n)</code>	Lecture robuste de n octets exacts depuis le socket
<code>traiter_client(conn, addr)</code>	Handler complet : réception → déchiffrement → vérification → sauvegarde

Configuration :

```

HOTE    = "0.0.0.0"          # Toutes les interfaces
PORT    = 9443
DOSSIER = "received_files"  # Répertoire de sortie

```

Nommage des fichiers sauvegardés :

```

received_files/20260513_160234_mon_fichier.txt
                |         |
                |         |
            Horodatage   Nom original

```

7.3 `crypto_utils.py`

Rôle : Couche cryptographique pure. Toutes les opérations crypto sont des appels `subprocess` vers OpenSSL CLI.

Fonctions :


```

chiffrer_fichier_aes(fichier_entree, fichier_sortie)
    → Génère clé (32B) + IV (16B) aléatoires
    → openssl enc -aes-256-cbc -nosalt
    → Retourne (clé, iv)

dechiffrer_fichier_aes(fichier_entree, fichier_sortie, cle, iv)
    → openssl enc -d -aes-256-cbc -nosalt
    → Opération inverse du chiffrement

chiffrer_cle_rsa(cle, iv, chemin_cle_publique)
    → Concatène cle (32B) + iv (16B) = 48 octets
    → openssl pkeyutl -encrypt -pkeyopt rsa_padding_mode:oaep
    → Retourne enveloppe binaire (~256 octets)

dechiffrer_cle_rsa(donnees_chiffrees, chemin_cle_privee)
    → openssl pkeyutl -decrypt -pkeyopt rsa_padding_mode:oaep
    → Extrait cle = donnees[:32], iv = donnees[32:48]
    → Retourne (cle, iv)

hash_sha256(fichier)
    → Lecture par blocs de 8 Ko
    → hashlib.sha256()
    → Retourne digest binaire (32 octets)

```

Choix architectural : OpenSSL CLI est utilisé plutôt que la bibliothèque Python

`cryptography` afin de rester proche des standards et d'éviter des dépendances tierces.

7.4 `demo.py`

Rôle : Démonstration locale du cycle complet chiffrement → déchiffrement, sans réseau.

7 étapes de la démo :

```

Étape 1 – Création d'un fichier de test temporaire
Étape 2 – Calcul du hash SHA-256 original
Étape 3 – Chiffrement AES-256-CBC
Étape 4 – Chiffrement RSA-OAEP de la clé
Étape 5 – Déchiffrement RSA-OAEP de la clé
Étape 6 – Déchiffrement AES-256-CBC
Étape 7 – Vérification des hash + contenu du fichier

```

Utilité : Permet de tester les fonctions cryptographiques sans démarrer le réseau. Idéal pour valider la PKI et les outils.

8. Sécurité du transport (TLS)

8.1 Configuration TLS

Côté serveur :

```
contexte = ssl.SSLContext(ssl.PROTOCOL_TLS_SERVER)
contexte.minimum_version = ssl.TLSVersion.TLSv1_2
contexte.load_cert_chain(
    certfile='pki/server/server.crt',
    keyfile='pki/server/server.key'
)
```

Côté client :

```
contexte = ssl.create_default_context()
contexte.minimum_version = ssl.TLSVersion.TLSv1_2
contexte.load_verify_locations('pki/ca/ca.crt')
contexte.check_hostname = True
contexte.verify_mode = ssl.CERT_REQUIRED
```

8.2 Poignée de main TLS



8.3 TLS vs chiffrement applicatif

Couche	Protocole	Protège
Transport (TLS)	TLS 1.2+	Canal réseau : écoute passive impossible
Application (AES+RSA)	AES-256-CBC + RSA-OAEP	Contenu : fichier chiffré même au repos

Les **deux couches** se complètent : TLS protège le canal, AES+RSA protège les données elles-mêmes.

9. Fonctionnalités implémentées

Cryptographie

- ✓ AES-256-CBC pour le chiffrement des fichiers
- ✓ RSA-2048 OAEP pour le chiffrement de la clé (enveloppe)
- ✓ SHA-256 pour la vérification d'intégrité
- ✓ Génération aléatoire de clé AES et IV à chaque envoi
- ✓ Chiffrement hybride (symétrique + asymétrique)

Infrastructure PKI

- ✓ Autorité de Certification (CA) auto-signée
- ✓ Certificat serveur X.509 signé par la CA
- ✓ Subject Alternative Name (SAN) pour localhost et 127.0.0.1
- ✓ Export de la clé publique du serveur
- ✓ Vérification de la chaîne de certification
- ✓ Script de génération automatisée (`setup_pki.sh`)

Transport sécurisé

- ✓ TLS 1.2 minimum obligatoire
- ✓ Validation du certificat serveur par le client
- ✓ Connexion TLS avec vérification du hostname

Transfert de fichiers

- ☒ Envoi de fichier depuis le client avec chiffrement complet
- ☒ Réception et déchiffrement sur le serveur
- ☒ Protocole binaire avec entêtes de taille
- ☒ Lecture socket robuste (buffering par blocs de 4 Ko)
- ☒ Sauvegarde avec horodatage (YYYYMMDD_HHMMSS)
- ☒ Vérification d'intégrité SHA-256 après réception
- ☒ Confirmation de réception envoyée au client

Outils de développement

- ☒ Démonstration locale sans réseau (`demo.py`)
- ☒ Vérification de la PKI avant exécution
- ☒ Affichage formaté des étapes et résultats
- ☒ Fichier de test inclus (`mon_fichier.txt`)

10. Fonctionnalités non implémentées

Le README documente explicitement les limitations de ce projet académique :

Fonctionnalité	Justification
mTLS (TLS mutuel)	Le serveur ne demande pas de certificat client
Signatures numériques RSA-SHA256	Pas de non-répudiation
AES-GCM	Pas de chiffrement authentifié (AEAD)
PBKDF2	Pas de dérivation de clé depuis mot de passe
Authentification client	Pas de vérification d'identité du client
Chiffrement du nom de fichier	Le nom est transmis en clair dans le paquet
Compression avant chiffrement	Données envoyées sans compression
Chiffrement en flux (streaming)	Tout le fichier est en mémoire
Perfect Forward Secrecy	Clés RSA statiques
Protection anti-rejeu	Pas de numéros de séquence

11. Analyse de sécurité

Points forts

Aspect	Évaluation
Algorithmes	Choix modernes et standards (AES-256, RSA-2048, SHA-256)
Tailles de clés	Conformes aux recommandations actuelles
Mode RSA	OAEP (plus sûr que PKCS#1 v1.5)
TLS	Version minimale 1.2 imposée
IV	Généré aléatoirement à chaque chiffrement
Vérification intégrité	Hash calculé sur le fichier original
PKI	Chaîne de confiance CA → Serveur valide

Points à améliorer (pour un usage en production)

Risque	Solution recommandée
Absence d'authentification client	Implémenter mTLS
Pas de non-répudiation	Ajouter signatures RSA-SHA256
CBC non authentifié	Passer à AES-GCM (AEAD)
Clés RSA statiques	Utiliser ECDHE pour PFS
Fichiers temporaires	Utiliser <code>tempfile.mkstemp</code> sécurisé
Pas de rate limiting	Ajouter limitation par IP
Pas de logs d'audit	Journaliser les accès et erreurs

Modèle de menace couvert

Menace	Protégé ?
Écoute passive du réseau	✓ TLS + AES chiffrent le canal et les données
Altération des données	✓ SHA-256 détecte toute modification
Usurpation d'identité serveur	✓ Certificat vérifié via CA
Usurpation d'identité client	✗ Pas d'authentification client
Rejeu de paquets	✗ Pas de protection anti-rejeu
Compromission des clés RSA	✗ Pas de PFS

12. Guide d'utilisation

Prérequis

- Python 3.10 ou supérieur
- OpenSSL installé et accessible dans le PATH
- Bash (Git Bash sur Windows)

Étape 1 — Génération de la PKI

```
bash setup_pki.sh
```

Résultat attendu :

```
[OK] Création de la CA...
[OK] Génération clé serveur...
[OK] Création certificat serveur...
[OK] Export clé publique...
[OK] Vérification : pki/server/server.crt: OK
```

Étape 2 — Démarrage du serveur

```
python server.py
```

Résultat attendu :

```
[SERVEUR] En écoute sur 0.0.0.0:9443...
```

Étape 3 — Envoi d'un fichier (dans un autre terminal)

```
python client.py mon_fichier.txt
```

Résultat attendu :

```
[CLIENT] Connexion à localhost:9443...  
[CLIENT] Fichier envoyé. Réponse : OK : fichier reçu et verifie
```

Étape 4 — Vérification du fichier reçu

```
received_files/20260513_160234_mon_fichier.txt
```

Démonstration locale (sans réseau)

```
# Sur Windows avec support UTF-8  
python -X utf8 demo.py
```

Résultat attendu :

≡ Démonstration Transfert Sécurisé ≡

[Étape 1] Création du fichier de test...	OK
[Étape 2] Calcul du hash SHA-256 original...	OK
[Étape 3] Chiffrement AES-256-CBC...	OK
[Étape 4] Chiffrement RSA-OAEP de la clé...	OK
[Étape 5] Déchiffrement RSA-OAEP...	OK
[Étape 6] Déchiffrement AES-256-CBC...	OK
[Étape 7] Vérification des hash...	OK – Intégrité validée !

≡ Démonstration réussie ≡

Résumé

Le projet **Secured Transfer** implémente un système de transfert de fichiers sécurisé répondant aux besoins fondamentaux de la **confidentialité** (AES-256-CBC + RSA-OAEP), de l'**intégrité** (SHA-256) et de l'**authenticité du serveur** (TLS 1.2 + PKI X.509).

Les choix algorithmiques sont conformes aux standards modernes (NIST, PKCS, RFC), la séparation des responsabilités entre les modules est claire, et les limitations sont documentées de façon transparente, ce qui en fait un projet académique de qualité pour l'enseignement de la sécurité des communications.